

10th International Young Scientists Conference on Computational Science

Application of Neural ODE with embedded hybrid method for robotic manipulator control

Zoya Meleshkova*, Sergei Evgenievich Ivanov, Lubov Ivanova

ITMO National Research University (ITMO University) Saint Petersburg, 49 Kronverksky Pr., Russian Federation

Abstract

The paper presents an approach to machine learning involving the use of a hybrid method in Neural ODE. The authors consider a way of the Neural ODE application with an embedded hybrid method for intelligent manipulator control in a non-deterministic environment. To reduce computational resources, the number of layers of the neural network using ODE is reduced, the solution of which is carried out by a hybrid transformation method. The hybrid method includes normalizing the system, linear transformation, and polynomial transformation. Using the hybrid method in Neural ODE can increase the learning and computation speed. The transformation method has high accuracy with low computational complexity of the algorithm. As a result of applying the hybrid method, the original nonlinear system is transformed to a simpler form. To solve the simplified transformed system, the scheme of the numerical Runge-Kutta method of the third order with an adaptive step is used. The results of the computational experiment show an increase in the speed and accuracy of the calculation. The presented approach to implementing the hybrid method in Neural ODE allows faster training of the neural network of the world model on limited data obtained from the real environment. After training on the data generated by the model of the world, a robotic arm always successfully reaches the object during subsequent interaction with the real environment. This solves the problem of the limited ability of the agent to interact when learning with the real environment in robotic systems.

© 2021 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 10th International Young Scientists Conference on Computational Science

Keywords: machine learning, model-based reinforcement learning, neural ODE, hybrid method

* Corresponding author. Tel.: +7-964-364-5744; fax: +7-964-364-5744.

E-mail address: 3645744@gmail.com

1. Introduction

Recently, neural ordinary differential equations - Neural ODE are increasingly used in various applied problems and research. To solve the problem of computing resources in neural networks, the number of layers in the Neural ODE has been reduced, for which it is necessary to apply effective numerical solution methods. The paper presents a hybrid method for increasing the speed of learning and computation in Neural ODE. The paper investigates the application of the hybrid method for Neural ODE in the control problem of robotic systems. As a result of applying the hybrid method in Neural ODE, the training of the neural network of the world model on limited data obtained from the real environment is accelerated.

Consider the areas of application of Neural ODE to various systems in modern scientific articles. A lot of modern scientific papers are devoted to the study of new models of neural networks using ordinary differential equations - Neural ODE Networks. The study [1] presents a family of deep neural network models using an ordinary differential equation solver (ODE). The investigated models with a constant memory size allow adaptation of accuracy and speed. The paper shows the properties of models in neural networks with continuous depth. The study [2] is devoted to the issues of computational loads at the stages of training models of neural networks of ODEs. The paper analyses the performance depending on the depth and dynamics of the growth of neural networks. An adaptive learning algorithm for residual networks, LipGrow, is proposed to increase network depth and speed up learning. In the study [3], the problem of vulnerability of deep neural networks to attacks is considered. The resistance of image classifiers based on Neural ODE Networks to attacks is investigated and the greater resistance for Neural ODE to attacks is shown in comparison with modern residual neural networks. In the study [4], it was shown that one linear layer is sufficient in the Neural ODE model for use in problems of approximation of irreversible continuous functions. The work [5] investigates the stability properties of neural ODEs. Empirical studies show that Neural ODE is more robust to random Gaussian disturbances and attacks than convolutional neural networks. To create reliable deep networks, a time-invariant stable neural ODE (TisODE) is proposed. In [6], to eliminate the constraints associated with the topology of the input space, augmented neural ODEs with lower computational costs are introduced. The authors of the article [7] investigate the problem of the efficiency of solving ordinary differential equations in Neural ODE. Methods for finding balanced solutions are investigated on the example of Izhikevich's neural model. The second-order Runge-Kutta method is used to improve real-time performance. The authors of the article [8] propose to supplement the Neural ODE with a structure based on the ODE dynamics for greater modeling flexibility. The proposed approach allows for improved performance and increased accuracy compared to ResNet and Neural ODE. In the article [9], to solve the problem of learning in Neural ODE on large datasets, it is proposed to apply simpler dynamics associated with a large number of network layers. The proposed approach can significantly reduce the training time for Neural ODE. The authors of [10] developed the Exchangeable Neural ODE (ExNODE) module for practical use in recognition problems. Computational experiments have shown the effectiveness of the module. In [11], the authors proposed to apply a new scheme of the numerical Runge-Kutta method in Neural ODE. Comparative analysis showed reduced training time and improved performance of the Neural ODE modification. The paper investigates the application of Neural ODE to the practical problem of controlling drives of robotic systems [12]. However, the presented works do not study the effectiveness of using Neural ODE in control problems for complex robotic systems.

The study considers the actual problem of intelligent control of robotic systems in a non-deterministic environment using ordinary differential equations in neural networks. To eliminate nonlinear disturbances in the control system and compensate for mutual influences of drives, an intelligent control system based on Neural ODE is used. The neural network replaces a compensator with fine-tuning in the control system of the robotic system.

Figure 1(a) shows the structure of a Neural ODE based drive control system. Neural ODE performs a non-linear transformation of the input vector of control parameters to the output. The notation is introduced here: NCD - Neural ODE based neural control device, NNM is a neural network model of a control object with a neural ODE. The neural network model is tuned to the object by the output mismatch signal. The neural control device adjusts according to the speed control criterion when changing the parameters of the object and the environment.

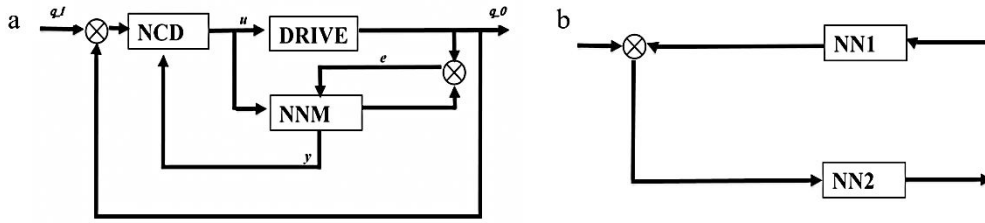


Fig. 1. (a) structure of the drive control system based on Neural ODE; (b) circuit of control of the robot manipulator.

Figure 1(b) shows the control scheme of the robotic system for solving the kinematics problem. Here NN1 is a neural network based on Neural ODE used to solve the direct problem of kinematics, NN2 is a neural network to solve the inverse problem of determining the generalized coordinates of links depending on the absolute coordinates of links. The tuning of the neural network model is performed by the mismatch of the input q^j and output q^o vectors of the generalized coordinates of the links. For a practical solution of the control problem using Neural ODE, the authors proposed a method described in the next section.

2. Method of solving the problem of intelligent control based on neural ODE

2.1. Model-based Reinforcement Learning with Deep Deterministic Policy Gradient (DDPG) and Dynamic Neural ODEs (DyNODE)

2.1.1. Markov decision-making processes

A Markov chain is a stochastic model that describes a sequence of possible events in which the probability of each event depends only on the state achieved in the previous event. The sequence in which the chain moves the state with discrete time steps gives a discrete-time Markov chain. Markov Decision Processes (MDP) are extensions of Markov chains; the difference lies in the addition of actions (providing choices) and rewards (rewards). Conversely, if for each state there is only one action (for example, "wait") and all rewards are the same (for example, "zero"), the process of making a Markov decision is reduced to a Markov chain. MDP is a framework that can solve many reinforcement learning problems. With the help of Markov decision process, the agent can come to the optimal policy for obtaining the maximum reward over time. MDP is based on the concepts of state, describing the current situation of the agent, the action that affects the dynamics of the process, and the reward observed for each transition between states. Solving the problem involves finding a policy that optimizes the performance criterion for the MDP in question. We consider the environment as a finite Markov discrete-time decision process M which is defined by S, A, f, r, γ, p_0 where S is a set of states, A is a set of actions, $s_{t+1} \sim f(s_t, a_t)$ transition distribution, $r: S \times A \rightarrow \mathbb{R}$ function rewards, γ is the discount factor, $p_0: S \rightarrow \mathbb{R}$ represents the distribution of the original state.

$$\max_{\varphi} J(\varphi) = \max_{\varphi} \mathbb{E} \left[\sum_t \gamma^t r(s_t, a_t) \right] \quad (1)$$

Thus, the goal of reinforcement learning is to find the policy $\pi_{\varphi}: S \times A \rightarrow \mathbb{R}$ in order to maximize the expected reward E . The policy π_{φ} is a neural network with parameters φ . The policy network chooses action based on current state and previous experience of interaction with an environment.

2.1.2. Model-based reinforcement learning

Unlike most research in deep reinforcement learning [28], reinforcement learning in real robotic systems such as robotic systems often face a number of limitations. One of the significant limitations for real-world reinforcement learning is the lack of a separate environment for learning and evaluating the agent's actions. All training data comes from a real robotic system, and an agent cannot have a separate exploration policy during training, since a separate training environment is not provided at no additional cost. Instead, the agent must perform well enough and operate safely throughout the training so as not to harm the real environment. For many systems, this means that the study

should be limited, and the resulting data have low variance, since a very small part of the set of states can be written to the memory buffer for further training of the agent on it. In addition, since there is often only one instance of a system, approaches that instantiate hundreds or thousands of environments to collect more data for distributed learning are usually incompatible with the real environment [24].

Another limitation is that all of these real-world robotic systems are either slow, fragile, or expensive enough to make the data they produce costly and policy research must be data efficient. In case there are autonomous system logs, they may not contain any data or the amount of data that modern reinforcement learning algorithms need.

In addition, learning in a real system can be time-consuming, as slower periods can range from one hour to several months (e.g., online ads, pharmaceuticals). Even in the case of high-frequency control problems, the learning algorithm must quickly learn from potential errors without having to repeat them several times before fixing them. Thus, learning in a real system requires the algorithm to be both selectively efficient and productive when working with the system.

Thus, in reinforcement learning without a model, there is no model of the environment with which the agent interacts; there is no need to study and model the inner workings of the system to assess the reward. In model-based reinforcement learning, we can define the reward function ourselves and calculate optimal actions using the model directly.

Model-based reinforcement learning increases the reward based on the performed actions. According to equations (2) and (1), the rewards depend on the policy network π with parameters θ and the dynamics of the system ρ (model):

$$\rho_\theta(s_1, a_1, \dots, s_T, a_T) = \rho(s_1) \prod_{t=1}^T \pi_\theta(a_t | s_t) \rho(s_{t+1} | s_t, a_t) \quad (2)$$

Earlier advances in deep reinforcement learning have been mainly in model-free approaches that did not attempt to learn a dynamic model, but rather used direct interaction with the environment to explore a policy or value function. However, their success in terms of performance comes at the cost of high data costs as they require a lot of interactions with the real environment. This can be impractical in robotic systems and thus greatly limits their application. By studying a model of a real environment and then using it as a replacement for a real system, model-based reinforcement learning algorithms can be significantly more efficient than their non-model counterparts. Based on this, we use model-based reinforcement learning in the experiment.

2.1.3. A model-based learning algorithm using a deep deterministic policy gradient

The Deep Deterministic Policy Gradient (DDPG) algorithm is a reinforcement learning algorithm that continually improves the Q-function (3) and the policy network π . First, we describe the DDPG deep deterministic policy gradient algorithm without a model, and then the DDPG based on the model.

To describe the system with which the agent interacts, a continuous space of parameters is used. The action space in the problem of moving objects is continuous and has large dimensions. Discretizing the action space to adapt some deep reinforcement learning techniques, such as deep q-learning (DQN) and policy gradient (PG), will result in a loss of precision and will require a lot of manual structure design effort to cope with a large number of parameter combinations in discrete space. Whereas deep policy gradient (DPG) uses deterministic policy to solve problems caused by multidimensional continuous action space, deep deterministic policy gradient (DDPG) is its variant using neural networks.

The classical DDPG algorithm uses two neural networks: the actor $\pi(s)$ and the critic $Q(s, a)$. The actor models the policy π , which maps the state s_t and the action a_t . The critic estimates the expected reward for the agent's action in the s_t state, which is trained using Bellman's equation (4), as in Q-learning, the data is fetched from the experience replay buffer:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma Q(s_{t+1}, \pi(s_{t+1})) \quad (3)$$

Here $r(s_t, a_t)$ is the reward that the environment gives when performing an action in a state. Actor $\pi(s_t)$ is trained to maximize the critic's score $Q(s_t, \pi(s_t))$. In other words, the actor defines the move for each state. The critic is optimized for a more accurate assessment of the expected reward.

However, we cannot train an agent with good performance using the original DDPG, because the agent cannot simulate a changing environment, and will train with limited data. A world model is a method that allows an agent to effectively understand the environment. For the purpose of the experiment, an additional neural network is created to simulate the dynamics of the environment using neural ODE. By interacting with the model, the agent will be able to

study the environment and effectively improve his policy. Further, we will refer to a DDPG with an actor that interacts with the modeled environment as a model-based DDPG.

Thus, in the problem of model-based reinforcement learning, an important role is played by the algorithm by which the model of the world is trained, since on its basis the system obtains predictions, which are then used for training. For a more accurate study and description of the dynamics of the world model, we will use dynamic neural ODEs using the transformation method, which will help evaluate not only the transition from the current state to the next, but the trajectory that is formed from the sequences of states.

2.1.4. Learning the dynamics of the world model using dynamic neural ODEs and the transformation method

In the model-based reinforcement learning problem, the DyNODE algorithm (Dynamic Neural Ordinary Differential Equations) is used. This algorithm is used to train the model of the world with which the agent interacts. The mechanism of dynamic neural ODE studies a dynamic model of the world in continuous control by integrating actions a into the structure of neural differential equations (NODE) during the propagation of errors in the values of the state s . In a basic neural network, tuple samples contain only two sequential states and action, and the loss function takes into account the predicted state for only one step. In the case of DyNODE, we sample the entire sequence of actions and propagate the predictions of state values through the dynamic NODE model, minimizing the prediction error along the entire length of the horizon. [16]

In the problem of achieving an object by a robotic manipulator, the observation of the system s is a 17-dimensional vector consisting of numerical values for describing the observations of the environment (observation), the achieved goal and the action.

Neural ODEs are a family of continuous time models that define the latent state $s(t_1)$. Let us consider the Cauchy problem:

$$\begin{cases} s'(t) = f(s, t, \theta) \\ s(t_0) = s_0 \end{cases} \quad (4)$$

Find $s(t_1)$ using the "solver", which we denote as ODEsolve:

$$s(t_1) = \text{ODESolve}(s(t_0), f, t_0, t_1, \theta) \quad (5)$$

In our method, as a "solver", we used a hybrid method for solving the ODE for the dynamics function $f_\theta(s(t), t)$ given by a neural network with parameters θ .

If we consider the dynamic function f_θ as a derivative of a state with respect to time, as in formula (6), we can use the ODE solver based on the previously described transformation method to obtain approximate values for the next state of the environment, taking into account the current state and action by integrating over a time interval.

$$\frac{ds(t)}{dt} = f(s(t), a(t), t, \theta) \quad (6)$$

One important assumption that needs to be taken into account in this numerical assessment is that the action remains constant throughout the trajectory in state-action space, that is, in the time interval between the current state and the next. This is true because the action is caused by foreign policy, and only then will the system evolve by a one-time step. Therefore, we can parameterize the derivative of the latent state with a weighted neural network and use numerical methods to compute an approximation of the next state s_{t+1} by integrating the following equation:

$$\hat{s}_{(t+1)} = s_t + \int_t^{t+1} \hat{f}(s_t, a_t, t, \theta) dt \quad (7)$$

Instead of making predictions from the current to the next state, we can extend the DyNODE model to include the entire sequence of actions, and try to predict not only the last step, but also intermediate ones, that is, the propagation of state values through the neural dynamics model of the ODE. This allows the model to better understand the transition function, to better generalize the studied dynamics to samples outside the training set compared to the canonical neural network, which only uses predicting the next state during training. [16].

For a practical ODE solution, the hybrid transformation method is described in the next section.

2.2. Method of Neural ODE Hybrid

Here is a hybrid method for solving neural ODEs. The transformation method has a high accuracy with a low computational complexity of the algorithm. In accordance with the method [13], the system is reduced to normal form, a linear transformation, and a polynomial transformation. To solve the simplified transformed system, the scheme of the numerical Runge-Kutta method of the third order with an adaptive step is used [11].

Assuming that the nonlinear function of the dynamics of neural ODEs can be represented by a polynomial, we write system (4) in normal form:

$$\dot{X} = CX + \sum_{|\nu|=2}^n c_\nu \prod_{i=1}^m x_i^{\nu_i} \quad (8)$$

Where X is the required vector of states for neural ODEs, C is a square matrix.

The vector index and norm are applied here: $|\nu| = \nu_1 + \nu_2 + \nu_3 + \dots + \nu_m$, $0 \leq \nu_i \leq n$.

The non-linear part is represented by an n degree polynomial. Under the conditions of Picard's theorem on the existence and uniqueness of a solution to the Cauchy problem, the right-hand side of system (9) is defined, continuous, and satisfies the Lipschitz condition.

Let's carry out a linear transformation:

$$\dot{Y} = LY \quad (9)$$

As a result, the linear part is converted to a diagonal form:

$$\dot{Y} = \Lambda Y + \sum_{|\nu|=2}^n d_\nu \prod_{i=1}^m y_i^{\nu_i} \quad (10)$$

The matrix Λ is diagonal $\Lambda = \text{diag}[\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_m]$

Next, a polynomial transformation is performed:

$$y_s = z_s + \sum_{|\nu|=2}^n k_\nu^s \prod_{i=1}^m z_i^{\nu_i}, \quad s = 1, \dots, 2k, \quad (11)$$

where the required transformation coefficients.

As a result of a polynomial transformation, we get a system of the form:

$$\dot{z}_s = \lambda_s z_s + \sum_{|\nu|=2}^n p_\nu^s \prod_{i=1}^m z_i^{\nu_i} \quad (12)$$

The transformed system is much simpler than the original one and contains an order of magnitude fewer nonlinear components in the polynomial. This greatly simplifies its solution and the computational complexity of calculating the right-hand sides of the equations.

For the numerical solution of the simplified transformed system, the scheme of the numerical Runge-Kutta method of the third order with an adaptive step is used [11].

The calculation formulas of the numerical Runge-Kutta method of the third order have the following form:

$$z'(q) = f(q, z), q_{n+1} = q_n + h_n, z_{n+1} = z_n + k_1/6 + k_2/3 + k_3/2, \quad (13)$$

Where q - are the complex coefficients of the transformed system.

$$k_1 = h_n f(q_n, z_n), k_2 = h_n f(q_n + h_n/2, z_n + k_1/2), k_3 = h_n f(q_n + 3h_n/4, z_n + 3k_2/4) \quad (14)$$

To calculate the adaptive step, we apply the calculation formulas of the form:

$$h_{n+1} = h_n \epsilon (\max |k_1 - k_3| / 18)^{-0.2} \quad (15)$$

Having obtained the solution $\{q_n\}$ by the transformation formulas (12), we find Y and for the representation of the solution in the initial variables we perform the inverse change to the linear one: $X = L^{-1}Y$. According to the presented method, neural ODEs are solved. The presented hybrid transformation method makes it possible to efficiently solve a neural ODE.

3. Computational experiment results

3.1. Experiment environment

The environment for the experiment is part of OpenAI Gym and uses the physics engine MuJoCo [27]. OpenAI's Fetch environments are based on the 7-DoF Fetch 2 manipulator, which has a two-fingered parallel grip [23]. The set of states for all tasks is completely observable and immediately estimated by the reward function $r = R(s, a, s')$

In all Fetch problems, the goal is 3D and describes the desired position of the object (or the end result to achieve). The rewards are rare and binary: the agent receives a reward of 1 if the object is in the target location (with a 5 cm tolerance), and 0 otherwise.

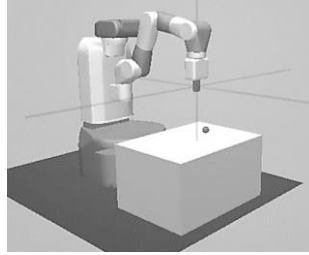


Fig. 2. Fetch environment for learning the operation of reaching an object (FetchReach-v1)

The actions are 4-dimensional: 3 dimensions define the desired movement of the gripper in Cartesian coordinates, and the last dimension controls the opening and closing of the gripper. Observations include the Cartesian gripper position, its linear velocity, and the robot's gripping position and linear velocity. If the object is present, we also include the Cartesian position and rotation of the object using Euler angles, its linear and angular velocities, and its position and linear velocities relative to capture. [18]

Achievement (FetchReach) - the task is to move the gripper to a given position. This task is very easy to learn and therefore provides a suitable benchmark to ensure that a new idea works.

3.2. Learning Algorithm

At the first stage, the agent performs random actions in order to fill the Real Replay Memory Buffer (RRMB) with tuples of the form s, a, s', r, d where s is the initial state, s' is the final state, a is an action, r is a reward, d - information about whether the episode is completed. In our experiment, an episode ends automatically after 200 failed attempts.

At the beginning of training, the vast majority of attempts to reach the object will be unsuccessful, since it is difficult to accidentally reach the object. Therefore, in order to quickly train the agent on what the environment gives a reward for, an algorithm HER (Hindsight Experience Replay [25]) is used, which significantly speeds up the training. At the first unsuccessful attempts the algorithm replaces the coordinates of the desired object with the coordinates to which the grip of the robotic arm came and reward it for this action.

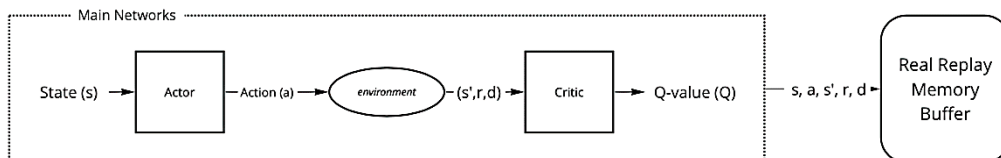


Fig. 3. The process of filling the memory buffer with data from the real environment

The agent world model is a neural network, and to use this world model in our training, we first need to train it on data from the RRBM. For training, we use the same data tuples s, a, s', r that has already been accumulated in memory.

After that, the world model can interact with the agent and train it on artificially generated Simulated Replay Memory Buffer (SRMB) tuples.

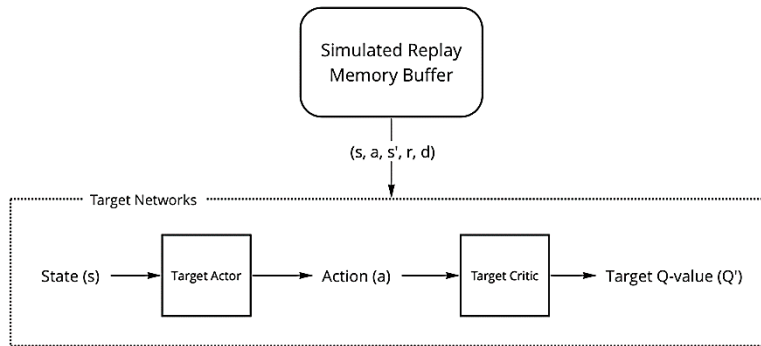


Fig. 4. Training the agent on the memory buffer generated by the neural network of the world model

After enough data has accumulated in the SRMB memory buffer to train the neural networks of the actor and the critic, the learning process begins, for which two additional neural networks are used.

Additional neural networks Target Actor and Target Critic are used to find the target Q' . A batch of data (B) is taken from the Simulated Replay Memory Buffer. For each tuple in the batch, there is a target value Q' . Once the gradients are found, a gradient descent is performed for the critic and a gradient descent for the actor.

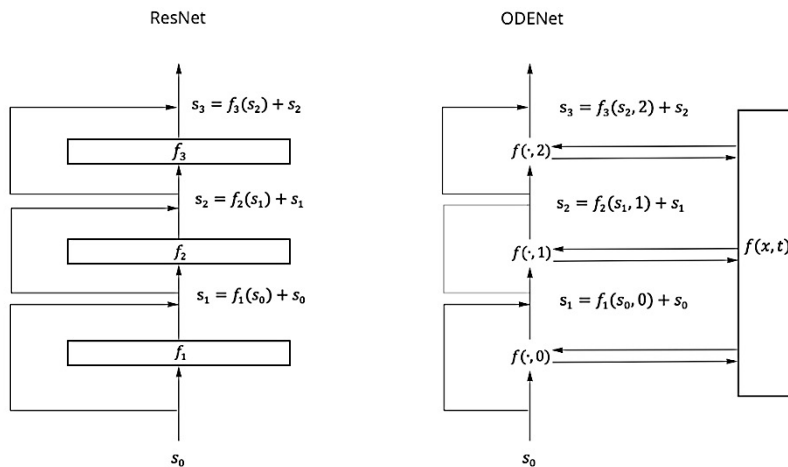


Fig. 5. ResNet and NeuralODE architecture

After the neural networks of the actor and the critic are trained on the data tuples generated by the world model, the agent can interact with the environment. Next, we will consider the results of interaction with the environment of an agent trained on a world model with the ResNet architecture, as well as compare the results obtained in the study of the hybrid method based on the Runge-Kutta of the third order with methods for solving ODEs such as Runge-Kutta of the 5th and 8th orders.

4. Results of Computational Experiment

In robotics tasks, we often have to work in conditions of limited interaction with the real environment while training. The goal of the research is to find a method that would be efficient even using the minimum set of data from interaction

with the environment. Let us compare the method of training a model using neural ODEs with the formation of a model based on ResNet.

Table 1 shows the results of training a neural network that simulates a model of the world. The training of the model took place over ten epochs, in each of which there were 50 episodes (iterations) of training a model of the world on real tuples (s, a, s', r, d) from memory buffer generated by the real environment. Table 1 shows the average results of the loss function for each epoch. The lower the score, the better the model learns.

NODE HYBRID, NODE Runge-Kutta of order 5, NODE HYBRID Runge-Kutta of order 8, showed approximately the same result at the first stages, however NODE Runge-Kutta of order 5, NODE Runge-Kutta of order 8 is significantly slower and less productive than the NODE HYBRID method proposed by the authors.

Let us compare the learning outcomes of the third epoch of learning methods based on neural ODEs with the method based on ResNet. The loss function for the methods based on neural ODE has an approximate value of 0.035, while the loss function for the method based on ResNet at the third epoch has a value of 0.31, which means a gap of about 10 times. Further in the table, we can observe that the method based on ResNet only by the tenth epoch will achieve the same result that the methods based on ODEs had already in the third epoch. As can be seen from the table, even at the 10th epoch, the error of models based on neural ODEs is still 2.6 times less than the model trained on ResNet.

Table 1. Model loss per Epoch FetchReach-v1.

Epoch #	Epoch 1	Epoch 2	Epoch 3	Epoch 4	Epoch 5	Epoch 6	Epoch 7	Epoch 8	Epoch 9	Epoch 10
NODE HYBRID	0,611	0,048	0,035	0,026	0,019	0,017	0,014	0,010	0,009	0,009
NODE Runge-Kutta of order 5	0,613	0,049	0,035	0,027	0,019	0,017	0,013	0,010	0,009	0,009
NODE Runge-Kutta of order 8	0,600	0,048	0,035	0,026	0,019	0,017	0,013	0,010	0,009	0,009
Model-Based ResNet	2,446	0,547	0,314	0,203	0,137	0,094	0,069	0,052	0,041	0,036

The figure shows a graph based on data from Table 1. The graph clearly shows how the method based on ResNet has a large gap with methods based on neural ODEs, in particular, the largest gap is with the method NODE HYBRID. After the fifth epoch the gap begins to close rapidly but does not reach the results of methods based on neural ODEs. After training on the world model, the agent begins to interact with the real environment and receive rewards. If the goal is achieved, the agent receives 1, if it does not reach 0. Interaction with the environment took place over ten epochs, in each of which 50 attempts were made to reach the object. The table shows the average result for the epoch. We see that methods based on the use of neural ODEs, including the NODE HYBRID, on average show a result of 1, which means that the agent almost always achieves its goal. While an agent trained on a model of the world using the ResNet architecture showed 20% more unsuccessful attempts on the first integrations of interaction with the environment.

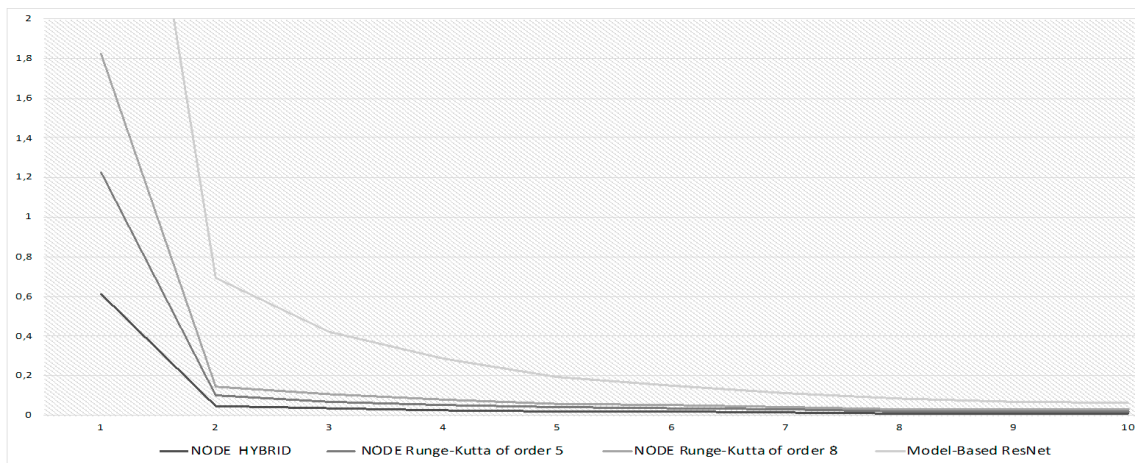


Fig. 6. Values of the loss function at each epoch

An agent trained on a model with ResNet achieves result 1 only by the seventh epoch, while methods based on Neural ODEs achieved this result already in the first epoch.

Table 2. Agent's rewards in interacting with the environment after training on the data generated by the world model

Epoch #	Epoch 1	Epoch 2	Epoch 3	Epoch 4	Epoch 5	Epoch 6	Epoch 7	Epoch 8	Epoch 9	Epoch 10
NODE HYBRID	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
NODE Runge-Kutta of order 5	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
NODE Runge-Kutta of order 8	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
Model-Based ResNet	0,82	0,91	0,92	0,95	0,98	0,99	0,99	1,00	1,00	0,99

Thus, the results of the study showed that the NODE HYBRID described in the article at the stage of training the model showed itself to be more productive compared to methods based on neural ODEs, but having other methods for solving NODE Runge-Kutta of order 5 and NODE Runge-Kutta of order 8. Compared to the results of training a model using a ResNet-based architecture, the NODE HYBRID method trains a world model several times faster and more accurately.

The agent, having trained on a model of the world, with weights adjusted using the NODE HYBRID, achieved a reward in almost all attempts, as well as other methods based on neural ODEs. While the agent trained on the model with the ResNet architecture began to reach the average score for the epoch equal to 1, only closer to the eighth epoch.

5. Conclusion

The study presents an approach to machine learning involving the use of a hybrid method in Neural ODE. The authors consider a way of the Neural ODE application with an embedded hybrid method for intelligent manipulator control in a non-deterministic environment. To reduce computational resources, the number of layers of the neural network using ODE is reduced, the solution of which is carried out by a hybrid transformation method. The hybrid method includes normalizing the system, linear transformation, and polynomial transformation. Using the hybrid method in Neural ODE can increase the learning and computation speed. The transformation method has high accuracy with low computational complexity of the algorithm. As a result of applying the hybrid method, the original nonlinear system is transformed to a simpler form. To solve the simplified transformed system, the scheme of the numerical Runge-Kutta method of the third order with an adaptive step is used.

The investigated models are evaluated based on the DDPG and DyNODE algorithm for training robots when the agent interacts with the OpenAI Gym FetchPush Environment virtual environment and the MuJoCo physical world simulator. Instead of making predictions from the current to the next state, the extended DyNODE model was used, which includes the entire sequence of the agent's actions, can predict not only the last step, but also intermediate ones, that is, the propagation of state values through the neural dynamics model of the ODE. This allows the model to better understand the transition function, to better generalize the studied dynamics on a sample outside the training set, compared to a neural network that only uses predicting the next state during training.

The method presented in the article more accurately characterizes the dynamics in continuous time and allows you to find a good policy with less real data. The neural network responsible for the model of the world is trained on a limited sample of data obtained from the real environment faster and more accurately than the model on ResNet. By the 10th epoch, the error of the world model trained on neural ODEs with the hybrid method is 2,6 times less than the error of the model trained on the canonical neural network with the ResNet architecture. At the same time, the model trained using neural ODEs achieves rewards in 100% of cases over ten epochs of interaction with the real environment, the model on ResNet reaches the object in about 82% of cases in the first epoch and reaches 100% only by the eighth epoch. The results of the computational experiment show an increase in the speed and accuracy of the calculation. The presented approach to implementing a hybrid method in Neural ODE allows you to train an agent on a world model, which requires significantly fewer data to train. The agent, after training on the data generated by the world model, achieves rewards faster than other considered approaches.

References

- [1] Chen, R. T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural ordinary differential equations. In *Advances in neural information processing systems* (pp. 6571–6583).
- [2] Dong, C., Liu, L., Li, Z., & Shang, J. (2020, November). Towards adaptive residual network training: A neural-ode perspective. In *International Conference on Machine Learning* (pp. 2616–2626). PMLR.
- [3] Carrara, F., Caldelli, R., Falchi, F., & Amato, G. (2019, December). On the robustness to adversarial examples of neural ode image classifiers. In *2019 IEEE International Workshop on Information Forensics and Security (WIFS)* (pp. 1–6). IEEE.
- [4] Zhang, H., Gao, X., Unterman, J., & Arodz, T. (2020, November). Approximation capabilities of neural ODEs and invertible residual networks. In *International Conference on Machine Learning* (pp. 11086–11095). PMLR.
- [5] Hanshu, Y. A. N., Jiawei, D. U., Vincent, T. A. N., & Jiashi, F. E. N. G. (2019, September). On robustness of neural ordinary differential equations. In *International Conference on Learning Representations*.
- [6] Dupont, E., Doucet, A., & Teh, Y. W. (2019). Augmented neural odes. In *Advances in Neural Information Processing Systems* (pp. 3140–3150).
- [7] Hopkins, M., & Furber, S. (2015). Accuracy and efficiency in fixed-point neural ODE solvers. *Neural computation*, 27(10), 2148–2182.
- [8] Zhang, T., Yao, Z., Gholami, A., Gonzalez, J. E., Keutzer, K., Mahoney, M. W., & Biros, G. (2019). ANODEV2: A coupled neural ODE framework. In *Advances in Neural Information Processing Systems* (pp. 5151–5161).
- [9] Finlay, C., Jacobsen, J. H., Nurbekyan, L., & Oberman, A. M. (2020). How to train your neural ode. *arXiv preprint arXiv:2002.02798*.
- [10] Li, Y., Yi, H., Bender, C. M., Shan, S., & Oliva, J. B. (2020). Exchangeable neural ode for set modeling. *arXiv preprint arXiv:2008.02676*.
- [11] S. Ivanov, A. Televnoy, T. Zudilova, T. Voitiuk Neural ODE Machine Learning Method with Embedded Numerical Method // 2021 28th Conference of Open Innovations Association (FRUCT). – IEEE, 2021.
- [12] Zhang, D., & Wei, B. (2017). A review on model reference adaptive control of robotic manipulators. *Annual Reviews in Control*, 43, 188–198.
- [13] Ivanov, S.E. A.V. Boukhanovsky, Hybrid transformation method for studying nonlinear models of dynamical systems with polynomial structure, *Izv. universities. Instrumentation*. 2019. T.62, № 8. C. 167—172.
- [14] Melnikov G.I., Ivanov S.E., Autonomization of nonlinear dynamic systems // *Scientific and technical bulletin of information technologies, mechanics and optics*. – 2014. – № 1(89). – 151–156 c.
- [15] Ramos, H., & Monteiro, M. T. T. (2017). A new approach based on the Newton's method to solve systems of nonlinear equations. *Journal of Computational and Applied Mathematics*, 318, 3–13.
- [16] Victor M. M. A., Rares R., Cristian G. F. (2020) DyNODE: Neural Ordinary Differential Equations for Dynamics Modeling in Continuous Control. URL <https://arxiv.org/abs/2009.04278>, cite arXiv:2009.04278
- [17] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W., 2016. Openai gym. URL: <http://arxiv.org/abs/1606.01540>. cite arxiv:1606.01540.
- [18] Plappert M. et al, (2018) Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research. URL: <https://arxiv.org/abs/1802.09464> cite arXiv:1802.09464
- [19] Ozel K., Yang Hu, Giovanni M., (2019) Reinforcement Learning for Robotic Manipulation using Simulated Locomotion Demonstrations. URL: <https://arxiv.org/abs/1910.07294> cite arXiv:1910.07294
- [20] A. Franceschetti, E. Tosello, N. Castaman, S. Ghidoni, (2018) Robotic Arm Control and Task Training through Deep Reinforcement Learning URL: <https://arxiv.org/abs/2005.02632> cite arXiv:2005.02632
- [21] J. Du, J. Futoma, F. Doshi-Velez, (2020) Model-based Reinforcement Learning for Semi-Markov Decision Processes with Neural ODEs. URL: <https://arxiv.org/abs/2006.16210> cite <https://arxiv.org/abs/2006.16210>
- [22] Y. Rubanova, R.T.Q. Chen, D. Duvenaud, (2019) Latent ODEs for Irregularly-Sampled Time Series. URL: <https://arxiv.org/abs/1806.07366> cite arXiv:1907.03907
- [23] Fetch Robotic Specification 07.28.2017 https://fetchrobotics.com/wp-content/uploads/2018/04/Fetch_robot_spec_overview.pdf
- [24] T. Wang et al., (2019) Benchmarking Model-Based Reinforcement Learning. URL: <https://arxiv.org/abs/1907.02057> cite arXiv:1907.02057
- [25] M. Andrychowicz, F. Wolski et al, (2017) Hindsight Experience Replay. URL: <https://arxiv.org/abs/1707.01495> cite arXiv:1707.01495
- [26] T. Hester et al., (2017/9) Deep Q-learning from Demonstrations. URL: <https://arxiv.org/abs/1704.03732> cite arXiv:1704.03732
- [27] E. Todorov, T. Erez, Y. Tassa, MuJoCo: A physics engine for model-based control – IEEE 2012
- [28] G. Dulac-Arnold, D. Mankowitz, T. Hester, (2019) Challenges of Real-World Reinforcement Learning. URL: <https://arxiv.org/abs/1904.12901> cite <https://arxiv.org/abs/1904.12901>
- [29] D. Silver et al (2014) Deterministic Policy Gradient Algorithms URL: <http://proceedings.mlr.press/v32/silver14.pdf>